TRS Winter School — Computer Vision Tasks

Parv Chopra

***Abstract*—This report documents two computer vision tasks completed as part of the TRS Winter School. Task 1 implements the RRT* (Rapidly-exploring Random Tree Star) algorithm for path planning on the IIT Kharagpur campus road network, extracting traversable roads from a map image via a multi-stage OpenCV preprocessing pipeline. Task 2 uses K-Means clustering to segment an image, identify the background, and replace it with a stock image, similar to a Zoom virtual background. Both tasks demonstrate practical applications of image processing, OpenCV, and algorithmic techniques.**

I. Introduction

Computer vision enables machines to extract meaningful information from images. This report covers two tasks that apply different aspects of computer vision:

1) **RRT* Path Planning:** Converting a campus map image into a traversable binary road mask through image processing, then applying the RRT* sampling-based motion planning algorithm to find an optimal path between two points.
2) **Background Removal via K-Means:** Using unsupervised clustering to segment image pixels into groups, identifying the background cluster, and replacing it with a stock background.

Both tasks rely heavily on OpenCV for image processing and NumPy for efficient array operations.

II. Task 1: RRT* Path Planning on IIT KGP Campus Map

A. Problem Statement

Implement the RRT* algorithm to find an optimal path between two designated points on the IIT Kharagpur campus map. The tree's exploration process must be visualized on the map image. Pre-processing with OpenCV is required to convert the map into a traversable format (identifying roads vs. obstacles).

Core topics: Path Planning Algorithms, OpenCV, Motion Planning.

B. Related Work

RRT* was introduced by Karaman and Frazzoli as an asymptotically optimal extension of the Rapidly-exploring Random Tree (RRT) algorithm [1]. While RRT finds *a* feasible path, RRT* continuously rewires the tree to find shorter paths, converging to the optimal solution as the number of samples approaches infinity.

Standard RRT* operates in open 2D/3D configuration spaces with scattered obstacles. Our setting—navigation on 1px-wide skeleton road masks—required significant adaptations (Section II-D).

C. Initial Attempts

Morphological opening for denoising: After HSV color thresholding, a 3×3 morphological opening was applied to remove noise. This was too aggressive—it eroded thin 1–3 px road lines, fragmenting the network from ∼23 connected components to 231 and destroying connectivity between start and end points.

Lesson: Standard morphological operations are unsuitable for structures thinner than the kernel. Gaussian blur + re-thresholding was adopted instead, as it averages pixel values without structural erosion.

D. Final Approach

The solution consists of two stages: (1) a multi-stage preprocessing pipeline that converts the campus map into a clean binary road mask, and (2) a skeleton-aware RRT* planner.

1) Preprocessing Pipeline: The pipeline converts a campus map image into a 1px-wide skeleton road mask through five stages:

1) **HSV Color Thresholding:** Road lines are blue-ish, which is isolated using HSV bounds $H \in [80, 150]$, $S \geq 10$, $V \geq 50$. HSV separates color from brightness, making it robust to lighting variations.
2) **Small Component Removal:** `cv2.connectedComponentsWithStats` with 8-connectivity removes components with area < 18 px (noise dots) while preserving contiguous road structures.
3) **Gaussian Blur Denoising:** A 3×3 Gaussian blur followed by re-thresholding at 127. Isolated white pixels surrounded by black are averaged below the threshold and disappear, while connected road lines survive.
4) **HoughLinesP Gap Filling:** `cv2.HoughLinesP` detects line segments and bridges gaps up to 5 px between disconnected road fragments. This runs *before* skeletonization because Hough requires multi-pixel-wide lines for reliable detection.
5) **Skeletonization:** Zhang-Suen morphological thinning (`skimage.morphology.skeletonize`) reduces all roads to 1px-wide center lines, producing a clean topology with no redundant parallel paths.

The pipeline order is critical: each stage depends on the previous one's output characteristics.

2) RRT Algorithm:* The RRT* planner builds a space-filling tree from a start point toward a goal, continuously optimizing path cost through rewiring.

Per-iteration loop:

1) **Sample** a random road pixel (or the goal with probability $p_{\text{goal}} = 0.05$).

TABLE I
PREPROCESSING PARAMETERS

Parameter	Value	Purpose
HSV range	(80,10,50)–(150,255,255)	Road color detection
Min component area	18 px	Remove noise dots
Hough threshold	12	Min votes for a line
Min line length	10 px	Filter short fragments
Max line gap	5 px	Bridge disconnected roads

2) **Find nearest** tree node via KDTree — $O(\log n)$.
3) **Steer** from nearest toward sample by `step_size` pixels, then **snap** to the nearest road pixel.
4) **Collision check** the edge on the dilated mask.
5) **Choose best parent** among nearby nodes within `rewire_radius` that minimizes cost.
6) **Rewire** — reroute nearby nodes through the new node if it reduces their cost.
7) If within `goal_radius` of the goal, record the path.

3) Skeleton-Aware Adaptations: Two key adaptations were required for 1px skeleton masks:

1. Snap-to-Road Steering: When RRT* steers toward a sampled point, the result almost never lands on a 1px line. The steered point is snapped to the nearest road pixel via a KDTree built over all road pixel coordinates:

$$p_{\text{new}} = \arg\min_{r \in \text{road}} \|p_{\text{steered}} - r\|_2 \tag{1}$$

2. Dilated Collision Mask: Two nodes on a curving skeleton may have a straight-line connection that briefly passes through a non-road pixel. A 2-iteration dilation creates ∼5px corridors for collision checking:

$$M_{\text{collision}} = \text{dilate}(M_{\text{skeleton}}, K_{3\times3}, \text{iter} = 2) \tag{2}$$

4) KDTree Data Structure: A **KDTree** (K-Dimensional Tree) is a binary space-partitioning data structure used to organize points in k-dimensional space. At each level of the tree, the points are split along one axis (cycling through dimensions), creating a balanced binary tree. This enables nearest-neighbor queries in $O(\log n)$ average time instead of $O(n)$ brute-force search.

In our implementation, two KDTrees are used:

- **Tree-node KDTree:** Built over all RRT* tree nodes. Used to find the nearest existing node to a newly sampled point. Rebuilt periodically as the tree grows.
- **Road-pixel KDTree:** Built once over all road pixel coordinates. Used for snap-to-road steering — finding the nearest road pixel to a steered point in $O(\log m)$ time (m = number of road pixels).

5) Optimizations:

- **Informed sampling:** Only samples from road pixels, never from obstacle space.
- **KDTree acceleration:** $O(\log n)$ nearest-neighbor via `scipy.spatial.KDTree`, rebuilt periodically.
- **Auto-computed parameters:** Step size, goal radius, rewire radius, and max iterations are all computed from `dim = max(width, height)`, enabling the same script to work on any map size.
- **Goal bias:** 5% of samples target the goal directly.

TABLE II
RRT* AUTO-COMPUTED PARAMETERS

Parameter	Formula	416px
Step size	$\max(3, \lfloor \text{dim} \times 0.005 \rceil)$	3
Goal radius	$\max(8, \lfloor \text{dim} \times 0.02 \rceil)$	8
Rewire radius	$\max(15, \lfloor \text{dim} \times 0.025 \rceil)$	15
Max iterations	$\max(50000, \text{dim} \times 100)$	50,000
Goal bias	0.05 (constant)	0.05

E. Results and Observations

The processing demo successfully finds a path between the designated start point $(15, 100)$ and end point $(380, 188)$ on the cropped campus map.

TABLE III
TASK 1 RESULTS

Metric	Value
Road mask coverage	∼7.5% of image
Goal reached	Yes (iteration ∼12,475)
Final path cost	∼437.2 px
Tree nodes at completion	∼50,000

An additional OSM-based demo validates the same RRT* planner on a larger 800×800 map downloaded from OpenStreetMap, reaching the goal at iteration ∼35,221 with a path cost of ∼1236 px.

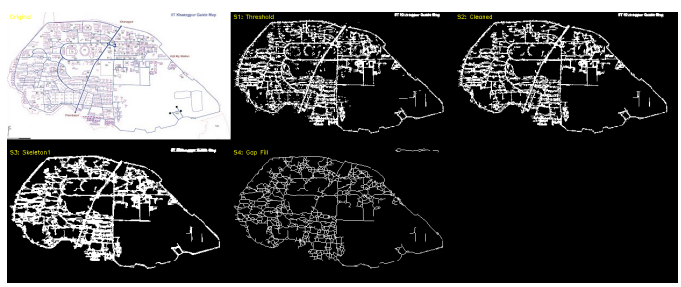

Fig. 1. Preprocessing pipeline stages: (left to right) HSV threshold, cleaned, denoised, gap filled, skeleton, final road mask.

F. Future Work

- **Bidirectional RRT*:** Growing trees from both start and goal simultaneously could dramatically reduce convergence time.
- **Weighted sampling:** Biasing samples toward under-explored regions of the road network.
- **Better image preprocessing:** Using adaptive thresholding or learned color models to improve road detection accuracy across varying map styles.
- **Custom line detection:** Developing a domain-specific line detection algorithm tailored to campus map road

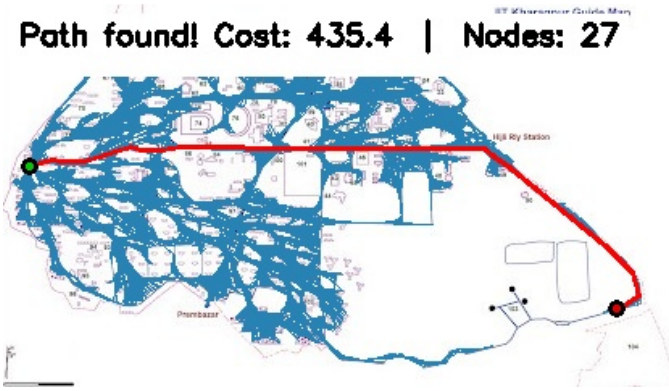

Fig. 2. RRT* result on the campus map. The red line shows the optimal path found between start and end points. The blue tree shows the explored nodes.

characteristics, rather than relying on generic HoughLinesP.

III. Task 2: Background Removal via K-Means Clustering

A. Problem Statement

Use K-Means clustering to segment an image, identify the background cluster, and remove it. Replace the removed background with a stock image (similar to a Zoom virtual background). Images with a significant, roughly uniform background should be chosen to ensure the background is identified as one of the largest clusters.

Core topics: Image Processing, OpenCV, Unsupervised Learning.

B. Related Work

K-Means clustering is a widely used unsupervised learning algorithm that partitions data into K groups by minimizing within-cluster variance. When applied to image pixels represented as color vectors, it effectively segments images by color similarity. Alternative approaches include semantic segmentation with deep learning and chroma keying (green screen removal).

K-Means offers a simple, interpretable baseline that requires no training data and works well when the background is roughly uniform in color.

C. Initial Attempts

Initial experiments used CIE LAB color space for clustering. LAB is perceptually uniform—equal Euclidean distances correspond to equal perceived color differences—making it a natural choice. However, for images where foreground and background share similar lightness values, HSV proved more effective at separating clusters by isolating the hue channel.

The initial version identified only the single largest border cluster as background. For images where the background spans multiple shades, this left residual background regions. Extending to the **two most border-dominant clusters** resolved this issue.

D. Final Approach

1) K-Means Clustering in Color Space: The image is Gaussian-blurred (5×5 kernel) to reduce noise, then converted to a perceptually meaningful color space. Each pixel becomes a 3D feature vector, and OpenCV's `cv2.kmeans` partitions all pixels into K clusters using K-Means++ initialization, run 10 times to avoid poor local minima.

The K-Means objective minimizes:

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2 \tag{3}$$

where C_i is cluster i and μ_i is its centroid.

2) Background Cluster Identification: A **border-dominance heuristic** identifies the background: pixels along all four image edges are collected, and the two cluster labels most frequent on the border are selected as background clusters. This works because background regions typically extend to the image edges.

3) Foreground Mask Generation: A binary mask is created where all pixels belonging to the two background clusters are set to 0 (background) and all others to 255 (foreground). The mask is then cleaned:

1) **Morphological closing** (elliptical 7×7 kernel, 3 iterations): Fills small holes inside the foreground.
2) **Morphological opening** (same kernel, 3 iterations): Removes small noise blobs.
3) **Gaussian feathering** ($\sigma = 5\,\text{px}$): A Gaussian blur is applied to the binary mask edges. This converts the hard 0/255 boundary into a smooth gradient, producing partial transparency at the foreground-background transition. Without feathering, composited edges appear jagged and unnatural; with it, the foreground blends seamlessly into the replacement background.

4) Compositing: The replacement background is resized to match the input dimensions. The final result is an alpha blend:

$$I_{\text{result}} = \alpha \cdot I_{\text{fg}} + (1 - \alpha) \cdot I_{\text{bg}} \tag{4}$$

where α is the feathered mask normalized to $[0, 1]$.

TABLE IV
Task 2 Configuration Parameters

Parameter	Value	Purpose
K	3	Number of clusters
Blur kernel	5×5	Pre-clustering noise reduction
Morph kernel	7×7	Mask cleanup
Morph iterations	3	Close/open iterations
Feather σ	5 px	Edge softening

E. Results and Observations

The pipeline successfully segments the input image, identifies the background via border dominance, and produces a clean composite with the replacement abstract background. The Gaussian-feathered mask creates smooth, natural transitions at foreground-background boundaries, avoiding the harsh cut-out look that a binary mask would produce.

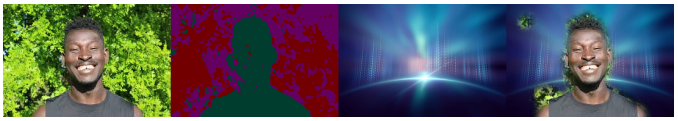

Fig. 3. Task 2 pipeline: (left to right) original image, K-Means segmented clusters, replacement background, final composite result.

Fig. 4. Foreground mask generated by K-Means background identification and morphological cleanup.

Removing the **two** most border-dominant clusters (rather than one) handles cases where the background spans multiple color shades, such as gradient backgrounds or shadows near edges.

Trade-offs:

- Increasing K can separate finer foreground details but may fragment the background into multiple clusters.
- $K = 3$ is default as it provides good separation for images with a dominant background.
- The border heuristic fails when the foreground subject touches the image border—manual cluster selection or spatial features would be needed in such cases.

F. Future Work

- **Spatial features:** Including (x, y) coordinates in the feature vector to make clustering spatially aware.
- **Adaptive K:** Using the elbow method or silhouette score to automatically determine K.
- **Boundary-aware detection:** Handling cases where the foreground subject touches or overlaps with image boundaries, where the border-dominance heuristic may misclassify foreground pixels as background.
- **Real-time processing:** Optimizing for video streams (frame-to-frame coherence).

IV. Conclusion

This report presented two computer vision tasks. Task 1 demonstrated a complete pipeline from raw map image to optimal path planning, combining multi-stage image preprocessing with a skeleton-aware RRT* algorithm. Key innovations included Gaussian blur denoising for thin structures and snap-to-road steering for skeleton masks. Task 2 showed how unsupervised K-Means clustering can be used for background segmentation and replacement, with border-dominance heuristics and morphological cleanup producing clean results comparable to commercial virtual background tools. Both tasks highlight the versatility of OpenCV and fundamental computer vision techniques in solving real-world problems.

References

[1] T. Erickson, "Robotic Path Planning: RRT and RRT*," Medium, 2019. [Online]. Available: https://theclassytim.medium.com/robotic-path-planning-rrt-and-rrt-212319121378